Rising Waters: AI-Powered Flood Prediction & Early Warning Web Application

Project Description:

Rising Waters is a Flask-based web application that helps residents of flood-prone regions assess their flood risk in advance and stay informed through timely, AI-assisted guidance. The platform combines a machine learning prediction engine, secure multi-mode authentication, and a Gemini-powered chatbot to deliver a complete early-warning experience from a single, easy-to-use interface.

The application supports both Google OAuth sign-in and manual signup with email and phone verification (powered by Brevo), so users can create an account through whichever method suits them best. Once logged in, users can enter local rainfall and river-level readings to receive a flood risk prediction, ask the built-in Gemini chatbot for safety guidance, and review their prediction history. The system is deployed on Render, with a PostgreSQL database handling user, verification, and prediction records.

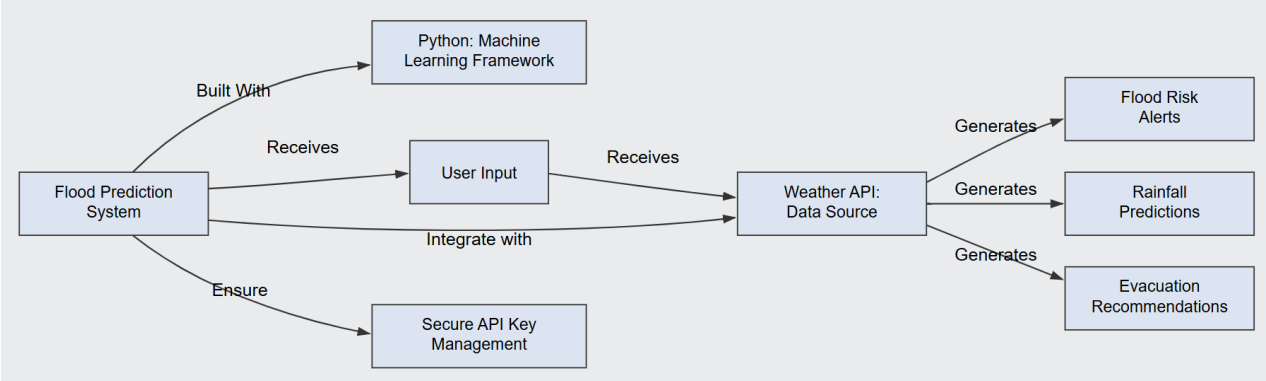
Scenarios

Scenario 1: A resident in a flood-prone village signs up using Google OAuth in seconds, then enters recent rainfall figures for their district. Rising Waters returns a flood risk level (Low / Moderate / High) with a plain-language explanation, all within seconds.

Scenario 2: A user without a Google account signs up manually with an email address and phone number. Brevo sends a verification email; after confirming their address, the user logs in and asks the Gemini chatbot what to do if the risk is rated "High" for their area, receiving step-by-step safety guidance.

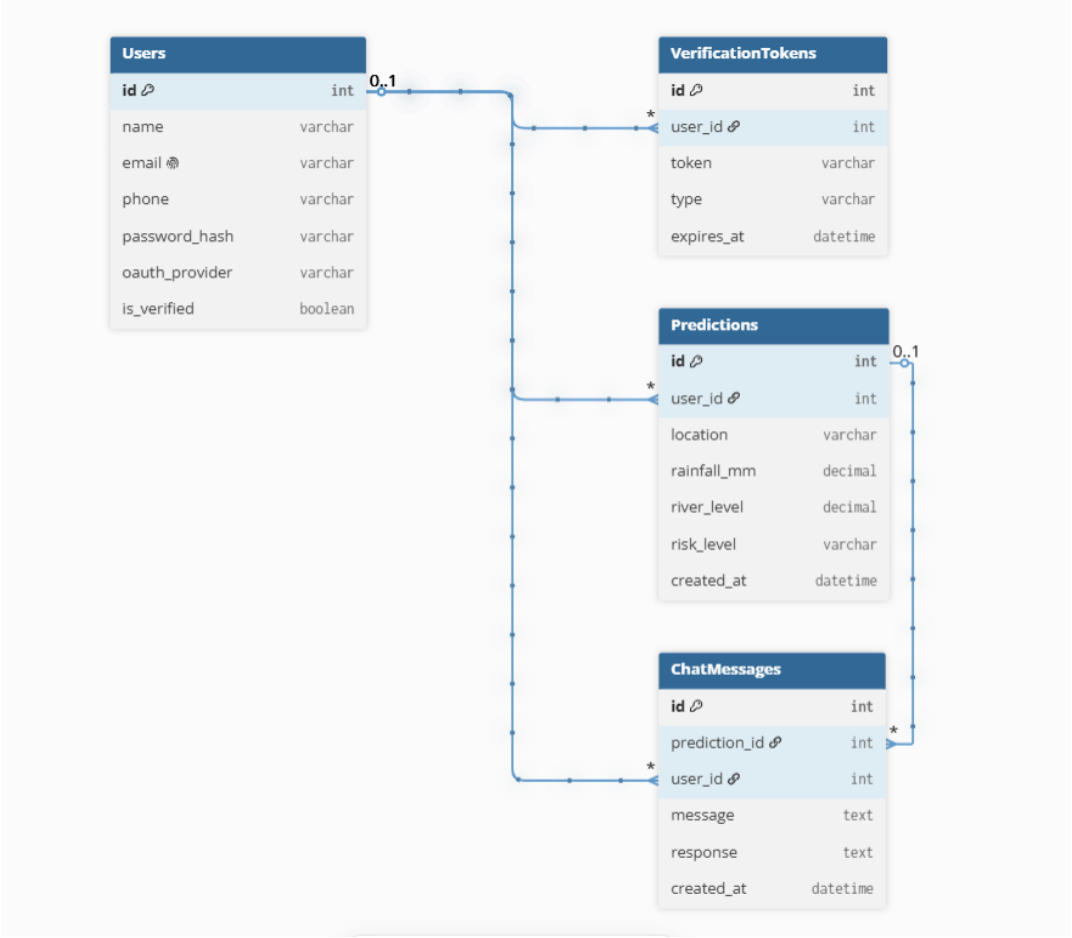
Scenario 3: A returning user checks the app during heavy monsoon rain, updates the river-level reading on the dashboard, and receives an updated prediction along with a chatbot suggestion to move valuables to higher ground given the rising risk trend.

Technical Architecture:



Description: Rising Waters uses a modular Flask architecture split across clearly separated concerns. The frontend renders responsive Jinja2 templates for authentication (auth/) and the main dashboard (main/). The backend is split into routes_auth.py, which handles Google OAuth callbacks, manual signup, and email/phone verification, and routes_main.py, which handles the flood-risk dashboard, prediction requests, and chatbot queries. prediction_engine.py houses the machine learning logic that turns rainfall and river-level inputs into a risk classification, while models.py defines the PostgreSQL schema via SQLAlchemy. The app integrates with Google OAuth for authentication, the Brevo API for transactional email, and the Gemini API for the chatbot, and is deployed on Render using render.yaml and a Procfile.

Note: *Rising Waters uses a PostgreSQL database, so an ER diagram is included below along with its description.*



ER Diagram Description: The Users table stores account details for both OAuth and manually-registered users, including a verification flag. Verification Tokens holds one-time email verification tokens linked to a user. Predictions store each flood-risk query a user makes, including the input readings, resulting risk level, and timestamp. Chat Messages logs the chatbot conversation history per user. Each of the three child tables has a one-to-many relationship back to Users.

Pre-requisites:

- Flask Framework Knowledge: Flask Documentation
- Google OAuth 2.0 Familiarity: Google Identity Documentation
- Brevo Transactional Email API: Brevo API Documentation
- Gemini API Familiarity: Gemini API Documentation
- PostgreSQL & SQLAlchemy: SQLAlchemy Documentation
- HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials
- Python Programming Proficiency: Python Documentation
- Version Control with Git: Git Documentation
- Render Deployment: Render Documentation

Project Workflow:

Milestone 1: Environment Setup and Data Preparation

- Activity 1.1: Set up the Python virtual environment and install all dependencies (Flask, scikit-learn, SQLAlchemy, psycopg, authlib, APScheduler, etc.).
- Activity 1.2: Collect and preprocess the flood prediction dataset — handle missing values, encode categorical features, and perform train-test split.
- Activity 1.3: Train and evaluate the Random Forest classifier; serialise the trained model to model.pkl using joblib.
- Activity 1.4: Configure the project directory structure: backend/, frontend/templates/, frontend/static/, uploads/models/.

Milestone 2: Database Design and Authentication

- Activity 2.1: Define SQLAlchemy models: User (with google_id, email_verified, pending_email, auth_method fields) and Prediction.
- Activity 2.2: Implement Flask-Login user loader and session management; write _run_light_migrations() for safe column additions on existing tables.
- Activity 2.3: Build dual-path authentication: Google OAuth 2.0 flow (Authlib) and manual signup with Brevo email verification and OTP-based email change.

Milestone 3: Core Feature Development

- Activity 3.1: Develop the Flood Risk Predictor route: parse form inputs, scale features, run model.predict_proba(), map output to risk categories, and store results.
- Activity 3.2: Integrate the Gemini AI Chatbot widget: server-sent events (SSE) or AJAX streaming endpoint that proxies user messages to the Gemini API.
- Activity 3.3: Implement the APScheduler severe-weather alert job: query saved locations, call weather API, compare thresholds, dispatch emails via Brevo.

Milestone 4: Frontend Development

- Activity 4.1: Design responsive Jinja2 templates for Home, Dashboard, Prediction Form, Results, Profile, and Auth pages.
- Activity 4.2: Build the chatbot widget UI with streaming text rendering, typing indicators, and a floating button launcher.
- Activity 4.3: Add interactive map component for saved-location selection and visualise historical prediction data using Chart.js.

Milestone 5: Deployment and Testing

- Activity 5.1: Configure Render Web Service: set build command (pip install + flask db upgrade), environment variables, and PostgreSQL add-on.
- Activity 5.2: Run full regression tests across auth flows (Google OAuth, manual signup, email verification) and prediction edge cases.
- Activity 5.3: Monitor Render logs, verify APScheduler starts only once (WERKZEUG_RUN_MAIN guard), and confirm light migration applies missing columns on first boot.

Milestone 6: Conclusion

Rising Waters is a Flask-based flood prediction and early-warning platform that brings together a machine learning risk model, flexible authentication (Google OAuth and verified manual signup), and a Gemini-powered chatbot in a single, accessible web application. The project, which spanned architecture design, authentication, prediction engine development, chatbot integration, frontend design, and deployment on Render, demonstrates how a focused combination of ML and generative AI can turn fragmented flood data into timely, actionable guidance for at-risk communities. Looking ahead, Rising Waters has clear room to grow: integrating a live weather API to remove manual data entry, adding SMS/email flood alerts, and expanding the prediction model with more diverse historical datasets. With continued refinement, the platform is well-positioned to evolve from a prediction tool into a comprehensive community flood-preparedness assistant.

Milestone 1: Environment Setup and Data Preparation

This milestone covers building the foundational infrastructure: the Python environment, dataset pipeline, and ML model. A well-prepared dataset and a validated model.pkl are the prerequisites for every feature built in later milestones.

Activity 1.1: Set Up the Development Environment

- Ensure Python 3.10+ is installed. Create and activate a virtual environment:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

- Core dependencies: Flask, flask-login, flask-mail, flask-wtf, sqlalchemy, psycopg, authlib, APScheduler, scikit-learn, joblib, google-generativeai, python-dotenv.

Activity 1.2: Data Collection and Preprocessing

- Source a historical flood dataset containing features such as rainfall (mm), river water level (m), soil moisture index, drainage capacity, elevation, and flood occurrence label.
- Handle missing values using median imputation; remove duplicate rows; encode binary flood label (0 = No Flood, 1 = Flood).
- Apply StandardScaler to numeric features; perform 80/20 stratified train-test split.

Activity 1.3: Train and Serialise the ML Model

- Train a RandomForestClassifier (n_estimators=100, random_state=42). Evaluate on the test set:

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier
```

- Serialise model and scaler: joblib.dump(model, "uploads/models/model.pkl") and joblib.dump(scaler, "uploads/models/scaler.pkl").

Activity 1.4: Configure Project Structure

- Organise the repository as follows:

```
backend/        — app.py, models.py, config.py, routes_*.py, alerts.py
frontend/       — templates/, static/css/, static/js/
uploads/models/ — model.pkl, scaler.pkl
.env            — SECRET_KEY, DATABASE_URL, GOOGLE_CLIENT_ID/SECRET,
                  GEMINI_API_KEY, BREVO_API_KEY, MAIL_* vars
```

Milestone 2: Database Design and Authentication

Milestone 2 establishes the data persistence layer and the dual-path authentication system. Correct schema design here prevents costly migrations later; the _run_light_migrations() pattern ensures new columns are safely added to existing production tables without Flask-Migrate overhead.

Activity 2.1: Define SQLAlchemy Models

- User model fields: id (UUID primary key), username, email, password_hash, phone, google_id, google_picture, email_verified, email_verification_token/expiry, reset_token/expiry, pending_email, email_change_code/expiry, role, created_at, last_login, is_active, auth_method.
- Prediction model fields: id, user_id (FK → users), rainfall_mm, water_level_m, soil_moisture, drainage_capacity, elevation_m, probability, risk_level, created_at.
- SavedLocation model: id, user_id (FK), label, latitude, longitude, created_at.

Activity 2.2: Light Migration Strategy

db.create_all() creates missing tables but does not add new columns to existing ones. The _run_light_migrations() function handles this safely in production:

```
def _run_light_migrations(app):
    from sqlalchemy import inspect, text

    try:
        inspector = inspect(db.engine)
        if 'users' not in inspector.get_table_names():
            return
        existing_cols = {c['name'] for c in inspector.get_columns('users')}
        needed = {
            'pending_email': 'VARCHAR(120)',
            'email_change_code': 'VARCHAR(10)',
            'email_change_code_expiry': 'TIMESTAMP',
        }
        with db.engine.begin() as conn:
            for col, coltype in needed.items():
                if col not in existing_cols:
                    conn.execute(text(f'ALTER TABLE users ADD COLUMN {col} {coltype}'))
    except Exception:
        app.logger.exception('Light schema migration failed (non-fatal).')
```

Activity 2.3: Authentication Implementation

- Google OAuth: Register app in Google Cloud Console; configure Authlib with server_metadata_url pointing to Google's OpenID configuration; handle /auth/google and /auth/google/callback routes.
- Manual Signup: Hash passwords with werkzeug.security; send a 6-digit verification token via Brevo SMTP on registration; enforce email_verified = True before login is permitted.
- Email Change Flow: Store new address in pending_email; generate OTP in email_change_code; confirm via /confirm-email-change; swap email fields on success.
- Profile Completion: before_request guard redirects authenticated users without phone or username to /complete-profile before any other endpoint.

Milestone 3: Core Feature Development

Milestone 3 builds the three headline features: ML flood prediction, Gemini AI chatbot, and the scheduled weather-alert system. Each feature is implemented as a Flask Blueprint to keep concerns separated and routes maintainable.

Activity 3.1: Flood Risk Predictor

- Accept POST form with fields: rainfall_mm, water_level_m, soil_moisture, drainage_capacity, elevation_m.
- Load scaler and model from disk; scale inputs; call model.predict_proba() to get flood probability.
- Map probability to risk levels: < 30% → Low, 30–60% → Moderate, 60–80% → High, > 80% → Critical.

```
features = np.array([[rainfall, water_level, soil_moisture, drainage, elevation]])
features_scaled = scaler.transform(features)
probability = model.predict_proba(features_scaled)[0][1]
risk_level = classify_risk(probability)
```

- Store result in Prediction table; render results template with probability gauge and contributing factor chart.

Activity 3.2: Gemini AI Chatbot Widget

- Configure google.generativeai with GEMINI_API_KEY from environment; use gemini-1.5-flash model with a system prompt specialised for flood safety guidance.
- Expose a /chat POST endpoint that accepts user message JSON and streams the Gemini response back to the frontend via SSE or returns complete JSON.
- Build a floating chatbot widget in JavaScript: renders a chat bubble, toggle button, and scrollable message history.

```
import google.generativeai as genai
genai.configure(api_key=os.environ["GEMINI_API_KEY"])
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content(user_message)
```

Activity 3.3: Severe Weather Alert Scheduler

- Use APScheduler BackgroundScheduler with an interval trigger (default 60 minutes, configurable via WEATHER_ALERT_INTERVAL_MINUTES env var).
- Guard against double-start in Flask debug mode: start only if not app.debug or WERKZEUG_RUN_MAIN == "true".
- For each SavedLocation, call an external weather API; if precipitation forecast exceeds threshold, build an alert email and dispatch via Brevo (flask-mail).

```
from apscheduler.schedulers.background import BackgroundScheduler
def init_scheduler(app, interval_minutes=60):
    scheduler = BackgroundScheduler()
    scheduler.add_job(check_alerts, "interval",
                      minutes=interval_minutes, args=[app])
    scheduler.start()
```

Milestone 4: Frontend Development

Milestone 4 focuses on building the user interface using Jinja2 templates, custom CSS with a responsive grid, and vanilla JavaScript for dynamic interactions. Every page inherits from a base.html layout that includes the navigation bar, chatbot widget, CSRF token, and flash message container.

Activity 4.1: Template Structure and Styling

- base.html: Master layout with navbar (login state aware), flash message block, chatbot widget embed, and footer.
- index.html: Landing page with hero section, feature cards, and a call-to-action linking to the prediction form.
- predict.html: Input form with labelled numeric fields, real-time character/range validation, and a submit spinner.
- result.html: Displays probability as an animated circular gauge, risk badge, contributing feature bar chart (Chart.js), and save/share options.
- dashboard.html: Lists past predictions in a sortable table; shows saved locations on an embedded map; links to alert preferences.
- CSS implements a clean card-based layout with CSS custom properties for colour tokens and supports dark/light mode via prefers-color-scheme.

Activity 4.2: Chatbot Widget UI

- Floating button (bottom-right) toggles a fixed-position chat panel with a header, scrollable message list, and text input.
- Messages are appended dynamically with distinct styles for user and AI turns; a typing indicator (three animated dots) appears while awaiting the Gemini response.
- Conversation history is kept in a JS array and sent with each request so Gemini maintains context across turns.

Activity 4.3: Data Visualisation

- Prediction result page renders a Chart.js horizontal bar chart showing feature importance scores from the Random Forest model.
- Dashboard renders a line chart of the user's historical prediction probabilities over time, giving a quick visual of risk trends.

Milestone 5: Deployment and Testing

Milestone 5 covers deploying Rising Waters to Render and validating the full system in production. Render provides a managed PostgreSQL database, persistent environment variable storage, and automatic HTTPS — making it the ideal platform for this Flask application.

Activity 5.1: Render Configuration

- Build Command: pip install -r requirements.txt (db.create_all() and _run_light_migrations() run at startup automatically — no separate migration step needed).
- Start Command: gunicorn "backend.app:create_app()" --workers 2 --bind 0.0.0.0:$PORT
- Environment Variables: Set SECRET_KEY, DATABASE_URL (Render Postgres internal URL), GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GEMINI_API_KEY, BREVO_API_KEY, MAIL_SERVER, MAIL_USERNAME, MAIL_PASSWORD, FLASK_ENV=production in the Render dashboard.
- Render PostgreSQL: Add a PostgreSQL add-on from the Render dashboard; copy the internal connection string to DATABASE_URL. Note: use TIMESTAMP (not DATETIME) for all datetime columns — SQLite accepts DATETIME but PostgreSQL does not.

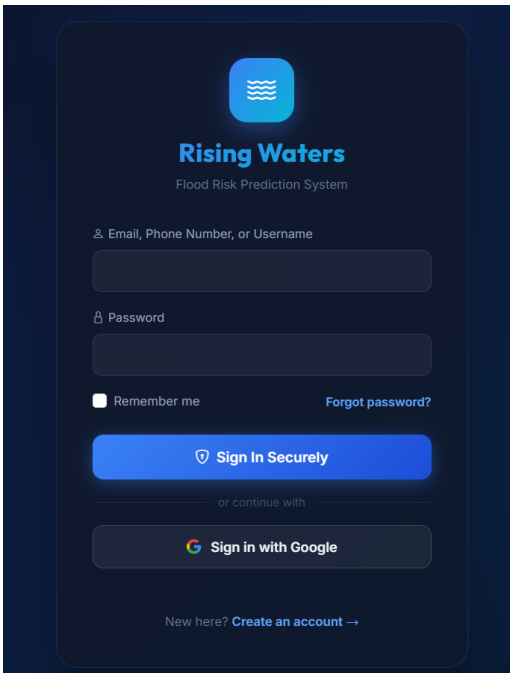
Activity 5.2: Local Testing Before Deployment

- Run python -m backend.app locally with a local PostgreSQL instance or SQLite for rapid iteration.
- Test all auth flows: Google OAuth redirect, manual signup → email verification → login, forgot password → reset link, email change → OTP confirmation.
- Submit prediction forms with boundary values (0 mm rainfall, extreme water levels) and verify risk level mapping.
- Trigger the alert scheduler manually and confirm email delivery through Brevo.

Activity 5.3: Production Validation

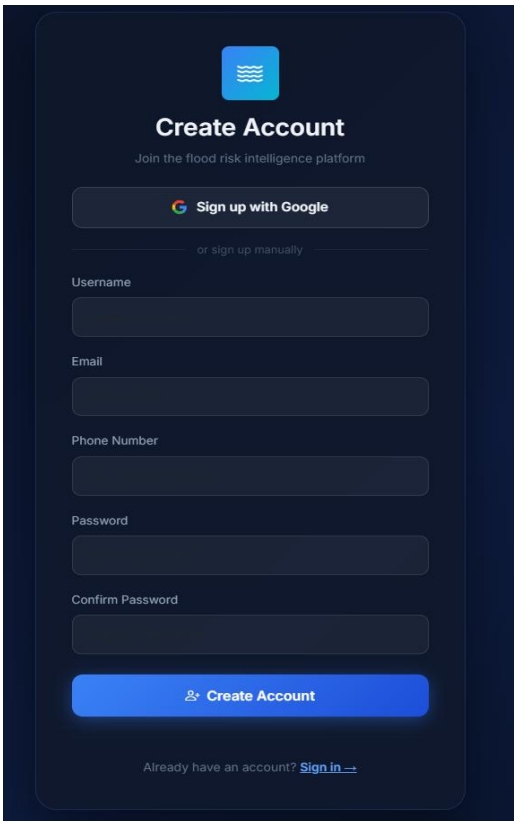
- After deploy, check Render logs for "Light schema migration" messages — confirms new columns were added successfully.
- Verify APScheduler starts with a single instance (no duplicate job logs) by checking WERKZEUG_RUN_MAIN handling.
- Test the chatbot widget end-to-end on the live URL; confirm HTTPS and OAuth redirect URIs are correctly whitelisted in Google Cloud Console.
- Run a full regression pass on the deployed site covering all six user-facing pages and the chatbot.

Exploring the Website's Web Pages:



Login Page:

Description: The Login page provides secure access to the Rising Waters platform using an email, phone number, or username with a password. Users can also sign in with Google, enable "Remember Me," or reset their password if needed.

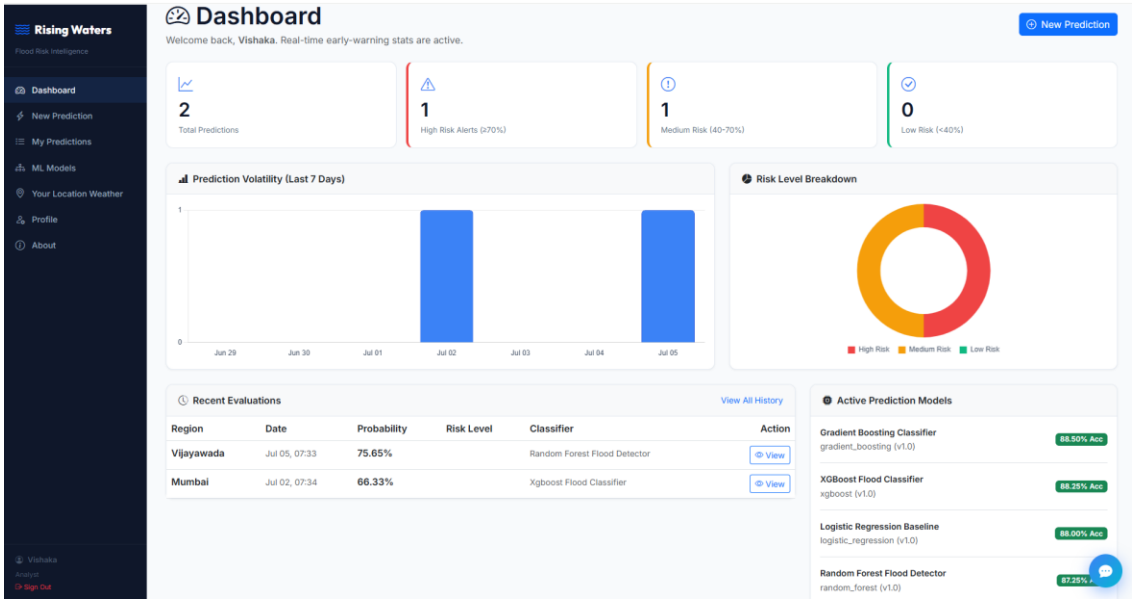


Create Account Page:

Description: The registration page allows new users to create an account by entering their username, email, phone number, and password. Users can also register quickly using their Google account for faster onboarding.

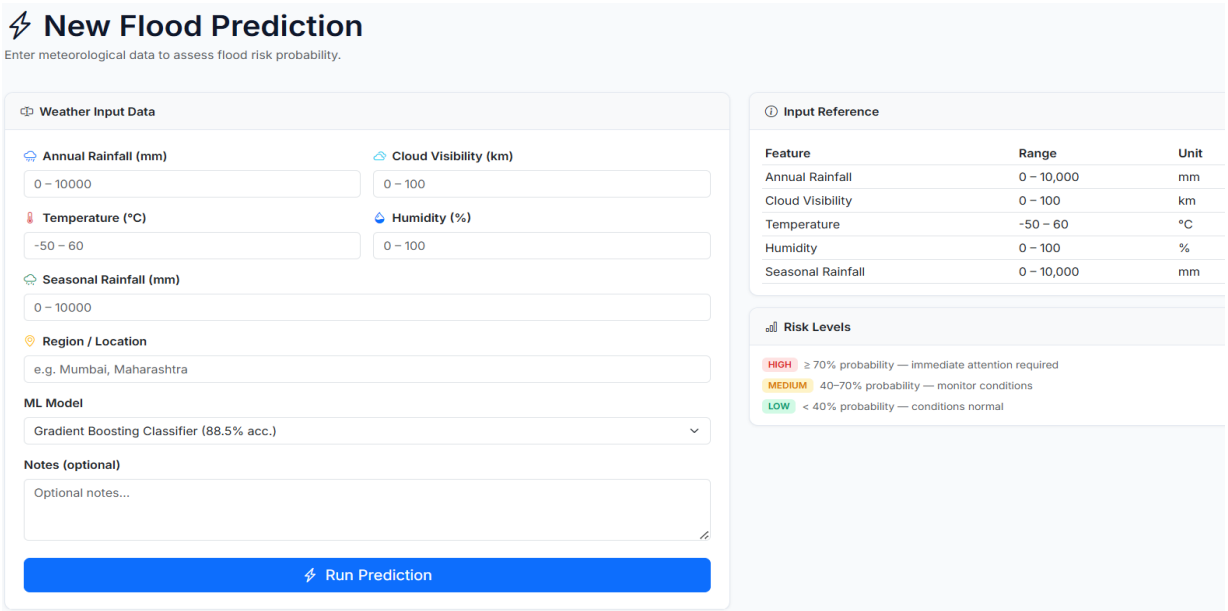
Dashboard:

Description: The Dashboard gives users an overview of flood prediction activities, including total predictions, risk-level statistics, recent evaluations, and model performance. Interactive charts help visualize prediction trends and risk distribution.



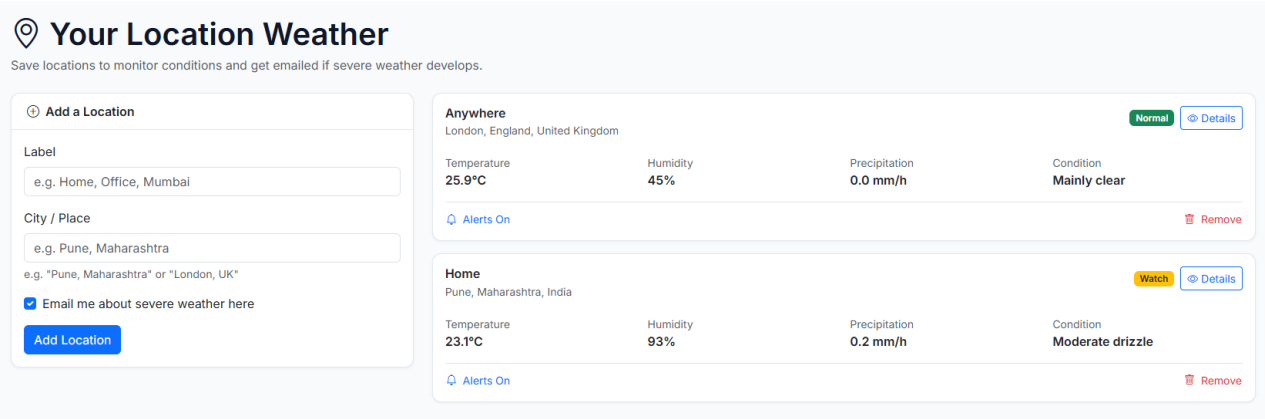
New Flood Prediction:

Description: This page enables users to enter meteorological parameters such as rainfall, temperature, humidity, and location to generate a flood risk prediction. Users can choose from multiple machine learning models to perform the analysis.



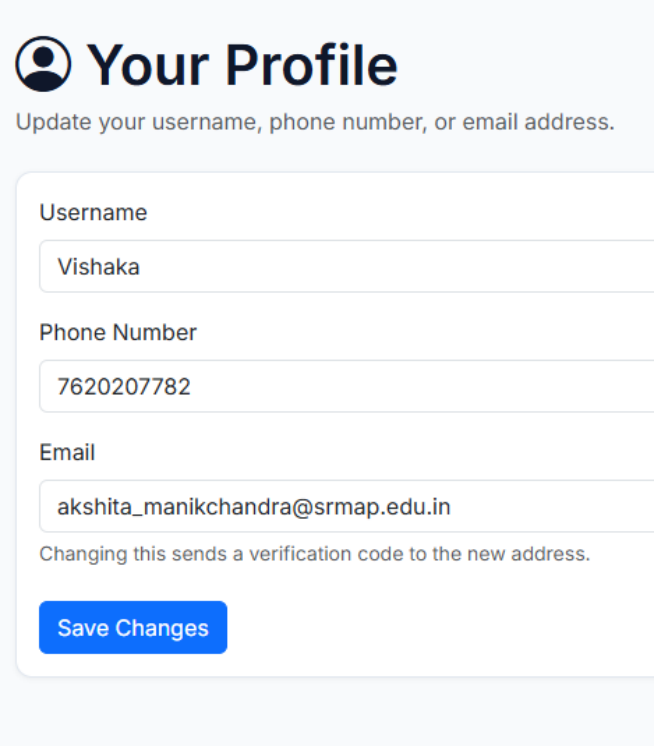
Your Location Weather:

Description: The Location Weather page allows users to save multiple locations and monitor real-time weather conditions. Email alerts can be enabled to notify users whenever severe weather or flood-related conditions are detected.



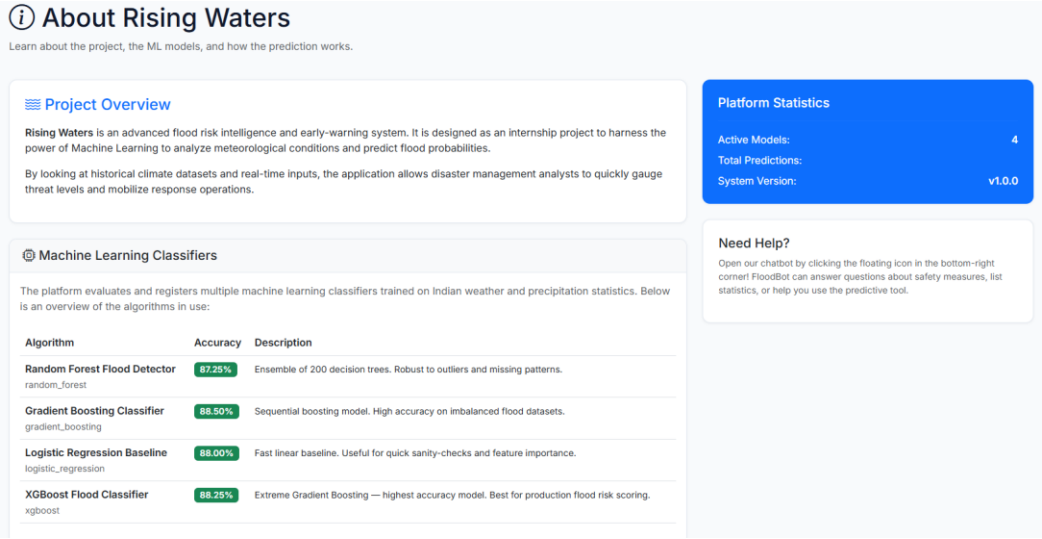
Profile:

Description: The Profile page allows users to update their personal information, including username, phone number, and email address. Any email change requires verification to maintain account security.



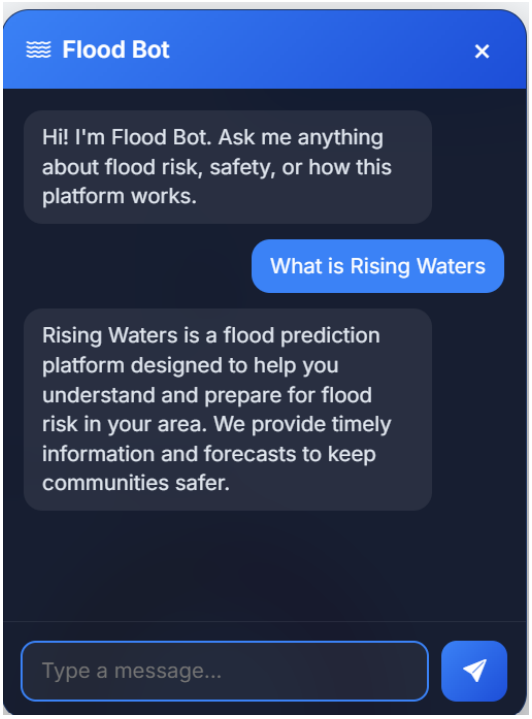
About Page:

Description: The About page explains the purpose of the Rising Waters platform, its flood prediction methodology, and the machine learning models used. It also displays platform statistics and provides quick access to user support.



Flood Bot (Chatbot):

Description: Flood Bot is an AI-powered assistant that answers user questions related to flood risks, weather conditions, and platform features. It provides instant guidance and helps users navigate the application efficiently.



Conclusion:

Rising Waters is a full-stack, production-deployed flood prediction and emergency management platform that demonstrates the practical integration of machine learning, generative AI, and real-time alerting within a secure web application. By combining a scikit-learn Random Forest model for flood probability prediction, the Google Gemini API for conversational guidance, and an APScheduler-driven severe weather alert system, the platform delivers actionable flood risk intelligence to users across all technical levels.

The project exercised a wide set of engineering disciplines: database schema design with safe migration strategies for PostgreSQL, dual-path authentication (Google OAuth and manual signup with Brevo email verification), background scheduling with Gunicorn-safe process guards, and responsive Jinja2 frontend development with Chart.js data visualisation.

Looking ahead, Rising Waters offers significant opportunities for expansion — integrating satellite imagery analysis for terrain-based risk refinement, extending the alert system to SMS via Twilio, adding multi-language support for regional reach, and exposing a public REST API for third-party emergency management integrations. With its modular blueprint architecture and cloud-native deployment on Render, the platform is well-positioned to scale from a demonstration tool into a production emergency-preparedness service.